

Rapport de modification

École supérieure

Électronique

Laboratoire POBJ

Modification 1730_PortierWireless

Réalisé par :

Léo Mendes

À l'attention de :

Juan José Moreno

Philippe Bovey

Table des matières :

Modification 1730_PortierWireless	1
1 Cahier des charges.....	5
2 Journal de travail	5
3 Cahier de laboratoire tâche effectuée	6
3.1 Lire le rapport effectué par Ismaël Page	6
3.2 Lire modifications de Diogo Ferreira	6
3.3 Reprogrammez et faire fonctionner projet en état actuelle.....	6
3.4 Compréhension et vue d'ensemble du code et des objectifs	7
3.5 Mise en place des modifications : 2.1Algorithme - appairage entre différents modules.....	8
3.6 Mise en place des modifications : 2.2Fonction envoi msg -> module alimenté pour changement de pile	8
3.7 Mise en place des modifications : 2.3Fonction sonore et visuelle pour changement de la batterie	8
3.8 Mise à jour de la documentation technique avec Flowchart, machines d'état et tests	9
4 Implémentation de pairing	10
4.1 Logique de pairing :	10
4.1.1 Au démarrage	10
4.1.2 En fonctionnement appairé.....	11
4.2 Description des fonction de PAIRING	12
4.2.1 Référence du fichier Pairing.c	12
4.2.2 Graphe des dépendances par inclusion de Pairing.c:.....	12
4.2.3 Énumérations	Erreur ! Signet non défini.
4.2.4 Fonctions	12
4.2.4.1 Serial_IsPairingRunning	12
4.2.4.2 NVM_OpenDriver	12
4.2.4.3 SaveSerialHarmony.....	12
4.2.4.4 LoadSerialHarmonyCharge.....	12
4.2.4.5 SavePairedSerialToFlash	12
4.2.4.6 LoadPairedSerialAndApply	13
4.2.4.7 GetActifSerialNbr	13
4.2.4.8 ResetSerialList	13
4.2.4.9 ShowPairingSuccess	13
4.2.4.10 PairingReset_DoReset.....	13
4.2.4.11 PairingReset_DoorTask	13
4.2.4.12 PairingReset_BellTask.....	13
4.2.4.13 RF_SendWithSerial (uint8_t *payload).....	13
4.2.4.14 bool Serial_CheckIfFromPaired (uint8_t *msg).....	13
4.2.4.15 uint32_t PairingManagement	13
4.2.5 Documentation du type de l'énumération	14
4.2.5.1 enum BellState	14
4.2.5.1.1 Valeurs énumérées:.....	14
4.2.5.2 enum DoorState.....	14
4.2.5.2.1 Valeurs énumérées:.....	14
4.3 Référence du fichier Pairing.h	15
4.3.1 Macros	15
4.3.2 Prototype de Fonctions	15
4.3.3 Variables global.....	15
5 Signature	15

1 Cahier des charges

Dans le cadre des modification du projet 1730_PortierWireless, plusieurs objectif on été définis :

- Ajout d'un système d'appairage entre les modules.
- Ajout de message spécifique pour changement des piles.
- Ajout d'une fonction sonore pour changement des piles.
- Test des fonctions implémentée.
- Documentations autours des modifications software effectuée.

2 Journal de travail

Journal de travail modification PortierWireless		
Jour de travail	Date	Avancements
1	24.02.2025	• Lecture des documentations du projet. • Essais sans réussite de démarrage des modules (manque câble et piles).
2	10.03.2025	• Compilation du programme fonctionnel. • Mise en place des adaptations IDC4 et alimentation forcée pour programmation.
3	24.03.2025	• Compréhension du code au moyen de Sourcetrail. • Création des fonctions de pairing.
4	07.04.2025	• Compréhension du code au moyen de Sourcetrail. • Implémentation des fonctions de pairing. • Débogage de la communication non fonctionnelle : les trames ne sont pas reçues par le système.
5	12.05.2025	• Changement de la logique d'envoi pour correspondre à la logique présente. • Débogage des communications.
6	19.05.2025	• Réussite du débogage des communications. • Mise en place de la logique de sauvegarde. • Débogage de la logique de sauvegarde.
7	26.05.2025	• Réussite du débogage de la sauvegarde NVM. • Mise en place de la validation des trames par envoi du numéro de série. • Débogage de la validation des trames par envoi du numéro de série. • Ajout de la réinitialisation du pairing. • Débogage de la réinitialisation du pairing.
8	02.06.2025	• Documentation : ◦ Doxygen ◦ Journal de travail ◦ Rapport de modifications • Tentative de débogage et de stabilisation : problèmes encore présents : ◦ La Door perd la sauvegarde du pairing ◦ Stabilité de la procédure de pairing ◦ Priorité du pairing : Bell d'abord, puis Door

3 Cahier de laboratoire tâche effectuée

3.1 Lire le rapport effectué par Ismaël Page

Lors de la première séance, le rapport original du projet a été lu. Il s'avère que de nombreuses différences ont été introduites depuis la première itération du projet afin de le rendre pleinement fonctionnel pour une utilisation réelle au sein de l'école. Cela rend le rapport moins pertinent pour la récupération d'informations concernant la partie software du projet, mais toujours utile pour la partie hardware.

3.2 Lire modifications de Diogo Ferreira

Les modifications proposées par Diogo Ferreira n'ayant pas abouti, j'ai analysé sa logique d'appairage, qui correspondait à ceci :

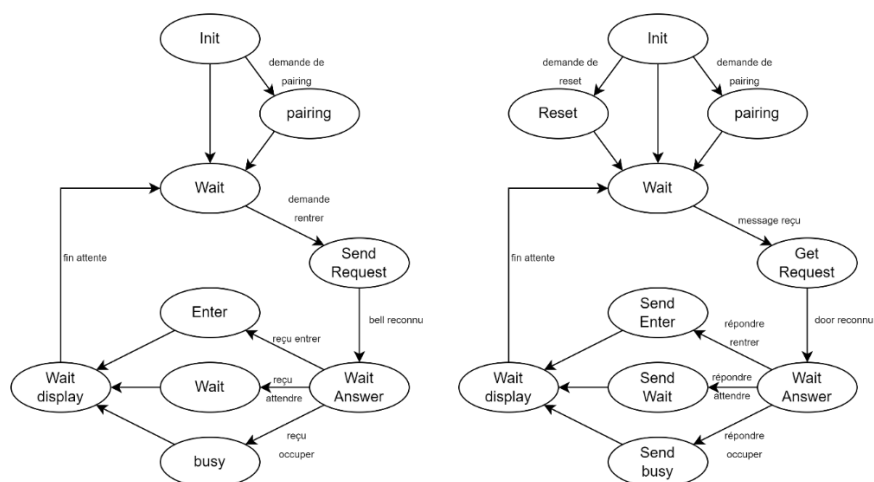


Figure 1- Logique pairing par Diogo Ferreira

L'implantation de cette procédure étant incomplète et peu détaillée dans le code, j'ai pris la décision de repartir d'une version stable, à savoir la version release de Monsieur Castoldi.

3.3 Reprogrammez et faire fonctionner projet en état actuelle.

Les tentatives de programmation ont été couronnées de succès sur le module Bell, mais totalement inefficaces sur le module Door indépendant. Cela s'explique par le fait que le module Bell n'est alimenté que lors de l'appui sur le bouton SW_RING, ce qui déclenche l'alimentation de la carte. Par conséquent, la carte ne peut être programmée que si ce bouton est maintenu enfoncé au moment précis de la programmation. Ce comportement peut convenir pour une simple programmation, mais il est totalement inadapté au débogage.

Cette contrainte m'a obligé à forcer manuellement l'alimentation de la carte via les broches GND et VCC. Toutefois, cette méthode empêche le fonctionnement normal du système : après une première communication et l'expiration du timeout, le système s'éteint automatiquement. Dès lors, seule une nouvelle pression sur le bouton permet de réalimenter la carte, ce qui rend impossible le débogage basé sur un envoi unique de trame par le module Bell.

En revanche, une fois les deux systèmes programmés sans débogage, les cartes assurent correctement et de manière stable leur fonction de base.

3.4 Compréhension et vue d'ensemble du code et des objectifs

Cette partie m'a pris énormément de temps en raison des nombreuses itérations de code, de la difficulté liée à la programmation du système et aux tests du fonctionnement. Le principe de deux systèmes parallèles basés sur un code unique était intéressant, mais difficile à représenter en détail. Afin d'améliorer ma compréhension, j'ai utilisé des logiciels tels que Sourcetrail pour représenter automatiquement le code de manière graphique.

Sourcetrail est un outil d'analyse statique qui permet de visualiser les dépendances entre fichiers, fonctions et variables dans un projet logiciel. Il génère automatiquement des graphes interactifs facilitant la navigation dans le code et la compréhension de sa structure, ce qui s'est révélé particulièrement utile pour un projet embarqué complexe.

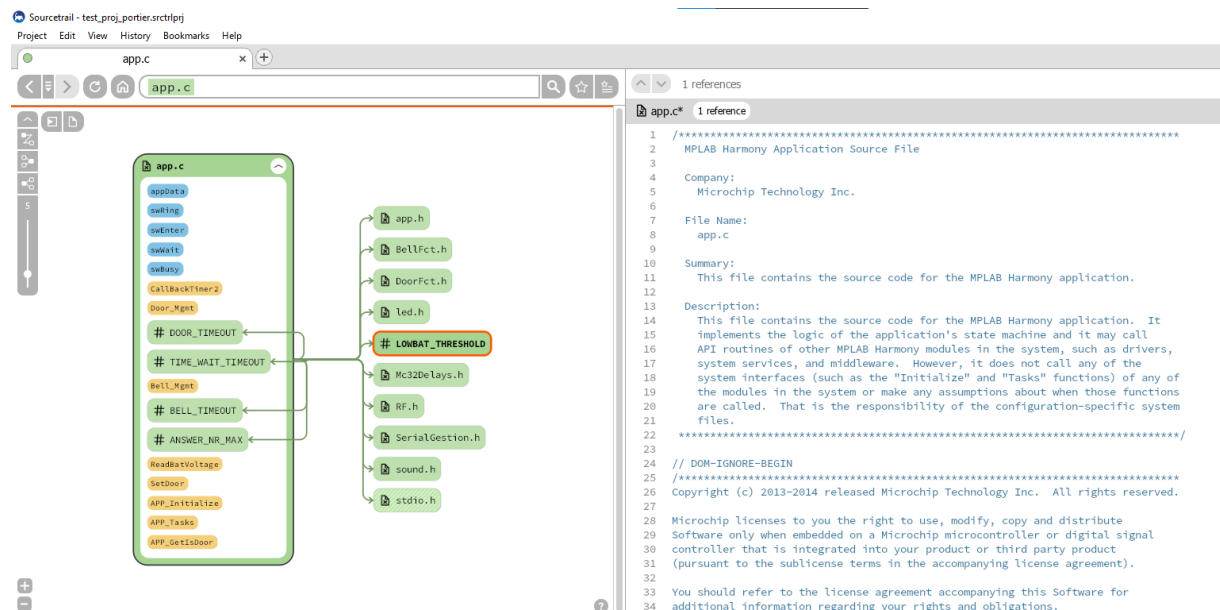


Figure 2 - Exemple SourceTrail

3.5 Mise en place des modifications : 2.1 Algorithme - appairage entre différents modules

3.5 Mise en place des modifications : Algorithme d'appairage entre modules

La mise en place de l'algorithme d'appairage a été effectuée conformément à la description du point 0. Cette implantation permet :

- Lancer un appairage au démarrage de manière manuelle, si aucun identifiant n'est présent en mémoire.
- Sauvegarder le numéro de série du module appairé dans la mémoire non-volatile (NVM).
- Valider, à la réception, que seules les trames provenant du module appairé sont traitées.
- Permettre un reset manuel du pairing via un appui prolongé sur les boutons définis.

Plusieurs fonctions ont été développées et intégrées à cette fin : gestion de l'identifiant, filtrage en réception, enregistrement permanent, et mécanisme de réinitialisation.

Cependant, malgré cette mise en œuvre, des problèmes persistent comme documenté dans le journal de travail :

- Le module Door perd parfois la sauvegarde du numéro de série après redémarrage, ce qui relance inutilement le mode appairage.
- Le module Bell ne permet le reset de son appairage que de manière non stable.
- La stabilité générale de la procédure d'appairage est encore insuffisante. Certaines séquences échouent ou ne se synchronisent pas correctement.
- La priorité logique de l'appairage (Bell initiateur, Door récepteur) n'est pas toujours respectée, ce qui génère des incohérences ou des blocages.
- L'implémentation actuelle utilise des sons déjà présents dans le système, ce qui limite la clarté du retour utilisateur. Des améliorations sont nécessaires pour distinguer les étapes du processus.

Ces problèmes ont été identifiés comme critiques pour l'usage du système en conditions réelles et nécessitent des corrections ultérieures.

3.6 Mise en place des modifications : Fonction sonore et visuelle pour changement de la batterie

Non effectué par manque de temps.

3.7 Mise en place des modifications : Fonction envoi msg -> module alimenté pour changement de pile

Non effectué par manque de temps.

3.8 Mise à jour de la documentation technique avec Flowchart, machines d'état et tests

L'essai de documentation avec Doxygen m'a permis d'effectuer une représentation correcte du fonctionnement du système.

Malgré tout, Doxygen nécessite une écriture précise des commentaires de fonctions, ce que je n'ai pas encore entièrement mis en place.

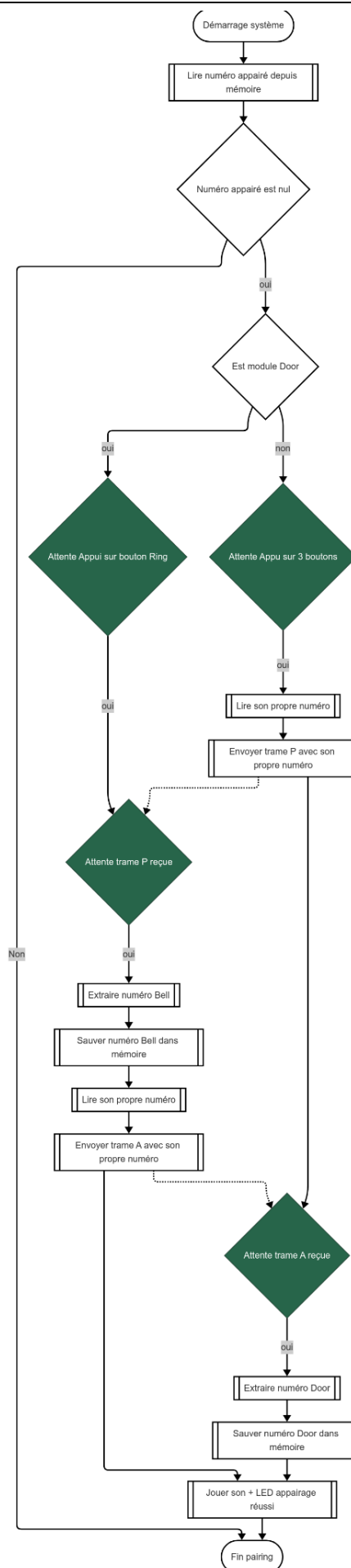
L'essai reste néanmoins concluant, m'ayant permis de générer un fichier LaTeX ainsi qu'un wiki HTML sur le projet, disponible à l'adresse suivante :

<https://1730portierwireless.neocities.org/>

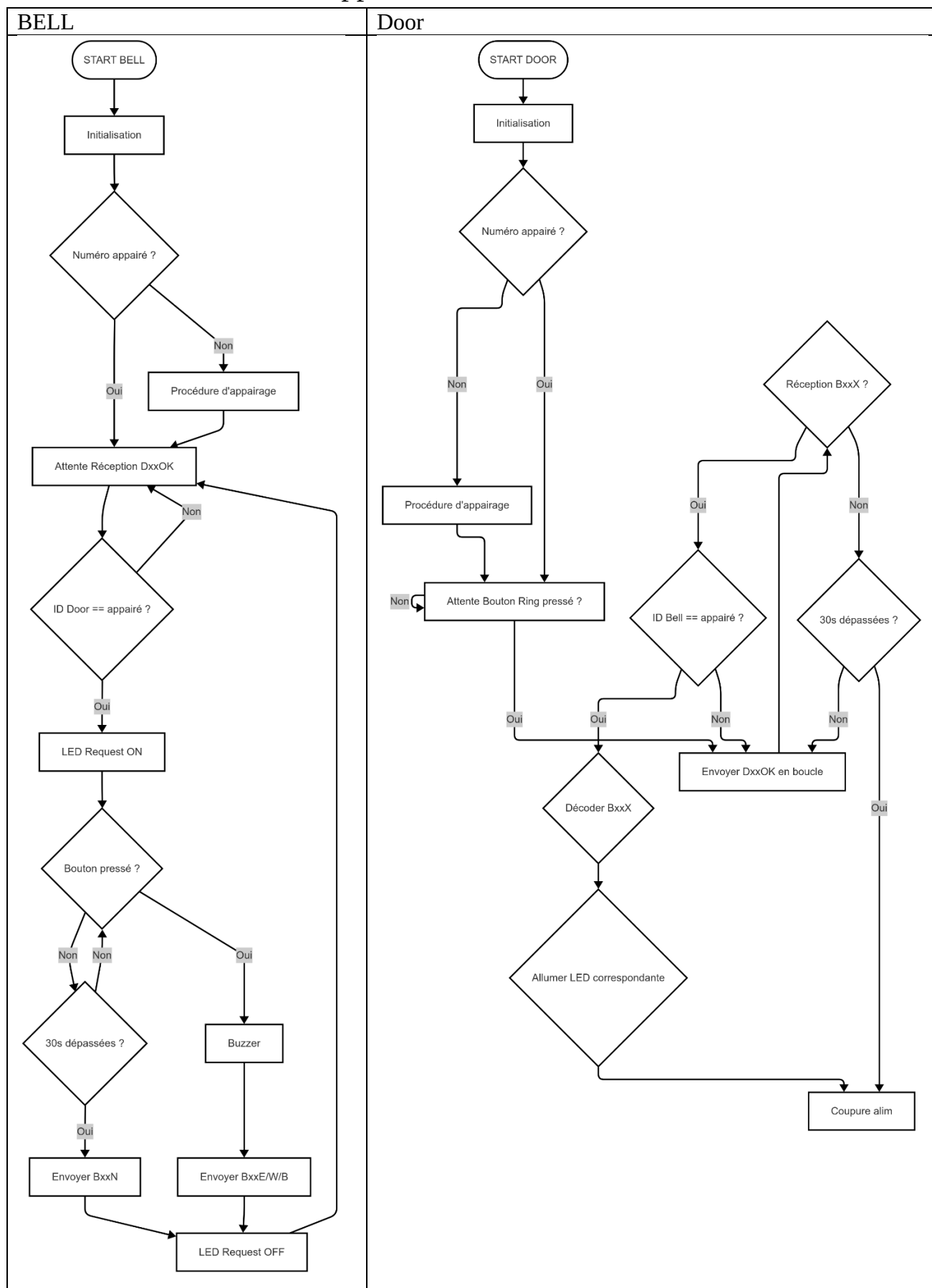
4 Implémentation de pairing

4.1 Logique de pairing :

4.1.1 Au démarrage



4.1.2 En fonctionnement appairé

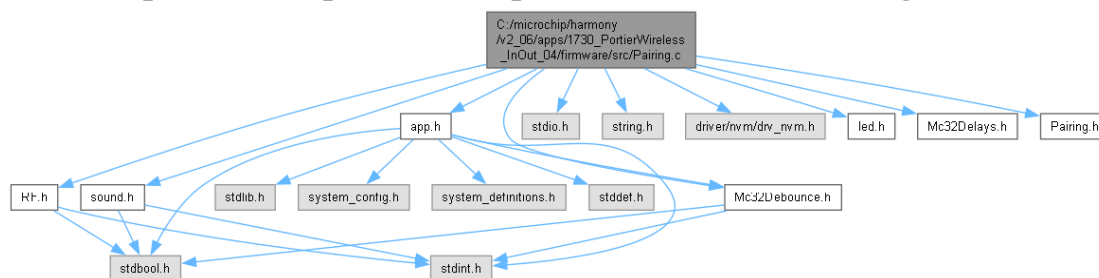


4.2 Description des fonction de PAIRING

4.2.1 Référence du fichier Pairing.c

```
#include "app.h"  
#include <stdio.h>  
#include <string.h>  
#include "driver/nvm/drv_nvm.h"  
#include "led.h"  
#include "Mc32Debounce.h"  
#include "Mc32Delays.h"  
#include "Pairing.h"  
#include "RF.h"  
#include "sound.h"
```

4.2.2 Graphe des dépendances par inclusion de Pairing.c:



4.2.3 Fonctions

4.2.3.1 Serial_IsPairingRunning

Indique si une procédure d'appairage est en cours.

Renvoie :

- true si l'appairage est en cours, false sinon.

4.2.3.2 NVM_OpenDriver

Ouvre le driver NVM si ce n'est pas déjà fait.

4.2.3.3 SaveSerialHarmony

Sauvegarde le numéro de série appairé en mémoire non volatile (NVM).

Paramètres entrées :

- Numéro de série à sauvegarder.

4.2.3.4 LoadSerialHarmonyCharge

le numéro de série appairé depuis la mémoire non volatile (NVM).

Renvoie :

- Le numéro de série chargé, ou 0 si non valide.

4.2.3.5 SavePairedSerialToFlash

Sauvegarde le numéro de série appairé courant en mémoire flash.

4.2.3.6 LoadPairedSerialAndApply

Charge le numéro de série appairé depuis la flash et l'applique à l'application.

4.2.3.7 GetActifSerialNbr

Retourne le numéro de série actuellement actif (appairé).

Renvoie :

- Le numéro de série appairé.

4.2.3.8 ResetSerialList

Réinitialise la liste des numéros de série et l'état d'appairage.

4.2.3.9 ShowPairingSuccess

Indique visuellement et auditivement le succès de l'appairage.

4.2.3.10 PairingReset_DoReset

Effectue la réinitialisation de l'appairage.

Cette fonction réinitialise la liste des numéros de série appairés, efface la mémoire flash correspondante, remet l'état d'appairage à zéro, et effectue une indication visuelle et sonore de la réinitialisation.

4.2.3.11 PairingReset_DoorTask

Tâche de surveillance du bouton d'appairage pour la porte (Door).

Déclenche une réinitialisation de l'appairage si le bouton swRing est maintenu appuyé suffisamment longtemps (appui long).

4.2.3.12 PairingReset_BellTask

Tâche de surveillance des boutons d'appairage pour la sonnette (Bell).

Déclenche une réinitialisation de l'appairage si les trois boutons (swEnter, swWait, swBusy) sont maintenus appuyés simultanément suffisamment longtemps (appui long).

4.2.3.13 RF_SendWithSerial (uint8_t *payload)

Envoie une trame RF avec le numéro de série, en utilisant P0 et P1.

4.2.3.14 bool Serial_CheckIfFromPaired (uint8_t *msg)

Vérifie si le message RF reçu provient du périphérique appairé.

4.2.3.15 uint32_t PairingManagement

Gère la procédure complète d'appairage côté Door ou Bell.

Cette fonction implémente la machine d'état d'appairage pour les deux rôles (Door/Bell). Elle pilote l'envoi/réception des trames, la sauvegarde du numéro de série appairé, l'affichage du succès, et la sortie de la procédure.

4.2.4 Documentation du type de l'énumération

4.2.4.1 enum BellState

4.2.4.1.1 Valeurs énumérées:

BELL_IDLE	Attente de l'appuis du bouton
BELL_SEND_P0	Envois première partie numéro de séries
BELL_SEND_P1	Envois deuxième partie numéro de séries
BELL_WAIT_ACK	Attente de la réponse de l'autre appareil
BELL_DONE	Pairing terminé

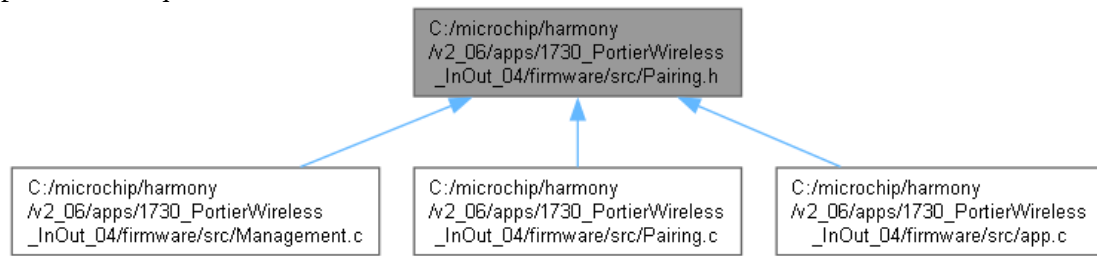
4.2.4.2 enum DoorState

4.2.4.2.1 Valeurs énumérées:

DOOR_IDLE	Attente de l'appuis du bouton
DOOR_WAIT_P	Envois première partie numéro de séries
DOOR_SEND_A0	Envois deuxième partie numéro de séries
DOOR_SEND_A1	Attente de la réponse de l'autre appareil
DOOR_DONE	Pairing terminé

4.3 Référence du fichier Pairing.h

Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :



4.3.1 Macros

- `#define NVM_SERIAL_BLOCK 0`
- `#define LONG_PRESS_TICKS 300`

4.3.2 Prototype de Fonctions

- `void LoadPairedSerialAndApply (void)`
- `void SavePairedSerialToFlash (void)`
- `uint32_t GetActifSerialNbr (void)`
- `void ResetSerialList (void)`
- `uint32_t PairingManagement (void)`
Gère la procédure complète d'appairage côté Door ou Bell.
- `void PairingReset_DoorTask (void)`
- `void PairingReset_BellTask (void)`
- `bool Serial_IsPairingRunning (void)`
- `bool Serial_CheckIfFromPaired (uint8_t *msg)`
Vérifie si le message RF reçu provient du périphérique appairé.
- `void RF_SendWithSerial (uint8_t *payload)`
Envoie une trame RF avec le numéro de série, en utilisant P0 et P1.
- `void ShowPairingSuccess (void)`
- `void TestFlashWriteRead (void)`

4.3.3 Variables global

- `S_SwitchDescriptor swRing`
- `S_SwitchDescriptor swEnter`
- `S_SwitchDescriptor swWait`
- `S_SwitchDescriptor swBusy`
- `uint32_t appPairedSer`

5 Signature

Lausanne, le 02.06.2025